

LC 416: Partition Equal Subset Sum

EPAM | FLIPKART | VISA

Problem Notes & Intuition

- We just need to return true or false depending on whether it is possible to make a split such that the sum of one subset equals the sum of the other subset.
- **Subarray vs Subset:** If it were a subarray problem, we could have used prefix sum, made a split at every possible position, and compared the sum from 0 to `split` with the sum from `split+1` to `n-1`.
- **DP Pattern:** But here we need **SUBSETS**. Elements can't be reused, and order doesn't matter. Hence, it is a **(take / skip) DP pattern**.
- **DP State:** `solve(idx, need)`
- **Initial Pass:** `solve(0, totsum / 2)`
- **Edge Case Check:** Add an initial check that `totsum` must be even. If it is odd, it's impossible to partition into two equal subsets, so return false.

C++ Solution

```

class Solution {
public:

    int n;
    vector<vector<int>> dp;

    bool solve(int idx, int need, vector<int>& nums) {
        if(need == 0) {
            return true;
        }

        if(idx == n) {
            return false;
        }

        if(dp[idx][need] != -1) {
            return dp[idx][need];
        }

        bool took = false;
        if(need - nums[idx] >= 0) {
            took = solve(idx + 1, need - nums[idx], nums);
        }

        bool skip = solve(idx + 1, need, nums);
        dp[idx][need] = took || skip;
        return dp[idx][need];
    }

    bool canPartition(vector<int>& nums) {
        n = nums.size();
        int totsum = accumulate(nums.begin(), nums.end(), 0);

        if(totsum % 2 == 1) {
            return false;
        }

        int need = totsum / 2;
        dp.assign(n, vector<int>((need + 1), -1));
    }
};

```

```
        return solve(0, need, nums);  
    }  
};
```